

End-to-End MLOps Pipeline

Binary Image Classification: Cats vs Dogs

Prepared by	Praveen Kanwar
Student ID	2024AC05746
Use case	Pet adoption platform - cat/dog classification
Technology stack	PyTorch, DVC, MLflow, FastAPI, Docker, GitHub Actions
Submission date	30 August 2026
Document status	Final submission report

A reproducible workflow covering data and code versioning, model training, experiment tracking, API packaging, containerization, CI/CD, monitoring, post-deployment evaluation, and submission evidence.

Executive Summary

This report documents the design, implementation, and verification of an end-to-end MLOps pipeline for classifying pet images as cats or dogs. The solution versions source code and data, trains and evaluates a reproducible PyTorch baseline, records experiments in MLflow, serves the promoted model through FastAPI, packages it in a Docker image, and automates testing and image publication with GitHub Actions. Docker Compose provides the deployment target, while structured logs and Prometheus metrics support operational monitoring.

The verified baseline achieved 75.48% accuracy on a held-out test set of 2,500 images. The latest continuous-integration run passed all 16 tests, verified the release model checksum, built the container, started the API, and completed both health and prediction smoke checks. The publish job pushed immutable commit and latest tags to GitHub Container Registry.

75.48%	16/16	24,998	4
Test accuracy	Tests passed	Valid images	API endpoints

Evidence boundary

CI and registry publication are verified by the successful GitHub Actions run. The repository contains the complete self-hosted Docker Compose CD workflow; this report does not claim a remote deployment run that has not been independently captured in the supplied evidence.

Table of Contents

Section	Page
Executive Summary	1
1. Problem Definition and Objectives	3
2. Architecture and Toolchain	3
3. M1 - Model Development and Experiment Tracking	5
4. M2 - Model Packaging and Containerization	7
5. M3 - CI: Build, Test, and Image Publication	7
6. M4 - Continuous Deployment	9
7. M5 - Monitoring, Logging, and Post-Deployment Tracking	9
8. Verification and Reproducibility	10
9. Rubric Compliance Matrix	11
10. Limitations and Recommended Next Steps	11
11. Conclusion	11
Appendices A-C	12

1. Problem Definition and Objectives

1.1 Use case

A pet adoption platform must classify an uploaded image as a cat or a dog. The system therefore implements binary image classification and returns both the predicted label and calibrated class probabilities through a REST interface.

1.2 Assignment objectives

- Version source code and the dataset so every result can be traced to an exact state.
- Train, serialize, evaluate, and track a baseline convolutional neural network.
- Expose repeatable health, readiness, prediction, and monitoring interfaces.
- Build, test, publish, and deploy a container image through automated pipelines.
- Collect operational telemetry and measure model performance after deployment.
- Package source, configuration, model artifacts, this report, and demonstration guidance for submission.

1.3 Acceptance criteria

Area	Acceptance evidence
Model	Serialized promoted model loads and predicts; test metrics and plots are retained.
Service	Health/readiness return success; prediction returns label and probabilities.
Container	Image builds reproducibly and runs as a non-root process.
CI/CD	Tests and smoke checks gate image publication and deployment.
Monitoring	Request count, error count, prediction count, and latency are observable.
Submission	ZIP, formal DOCX/PDF report, and <5-minute demonstration recording are present.

2. Architecture and Toolchain

2.1 End-to-end flow

1. Acquire the Kaggle Microsoft Cats vs Dogs dataset and version the raw snapshot with DVC.
2. Validate images, convert to 224x224 RGB, create deterministic stratified 80/10/10 splits, and version the processed output.
3. Train the baseline CNN, track parameters/metrics/artifacts with MLflow, evaluate the held-out test split, and promote a checksum-verified release model.
4. Serve the release model through FastAPI and package the application in a pinned Docker environment.
5. Run linting, unit/integration tests, integrity checks, a container build, and smoke tests on every change.
6. Publish successful main-branch images to GHCR and deploy the exact immutable image through Docker Compose.

7. Observe structured logs and Prometheus metrics, then evaluate simulated labeled requests for post-deployment quality.

2.2 Open-source stack

Lifecycle stage	Tooling	Purpose
Versioning	Git + DVC	Trace code, raw data, processed data, and release metadata.
Modeling	PyTorch + torchvision	Image preprocessing, augmentation, CNN training, serialization.
Tracking	MLflow	Log runs, parameters, metrics, plots, and model artifacts.
Serving	FastAPI + Uvicorn	Provide typed REST endpoints and API documentation.
Packaging	Docker + Compose	Create and operate a reproducible, isolated service.
Automation	GitHub Actions	Test, build, publish, deploy, and smoke-test changes.
Monitoring	Prometheus client + JSON logs	Measure volume, failures, predictions, and latency.

2.3 Repository organization

src/cats_dogs_mlops/	Application, data, model, and inference modules
scripts/	Training, evaluation, smoke, traffic, and packaging commands
tests/	Unit and integration tests
artifacts/	Promoted model, metadata, and evaluation plots
.github/workflows/	CI, image publication, and CD workflows
deployment/	Docker Compose deployment configuration
docs/	Architecture, evidence, checklist, and final report

3. M1 - Model Development and Experiment Tracking

3.1 Data and code versioning

Git records the complete project history, while DVC records dataset snapshots without placing large image collections directly in Git. The raw DVC stage references 25,005 files (approximately 1.69 GB). Image validation removes unreadable/corrupt inputs and produces 24,998 valid examples for model development.

Dataset item	Value
Source	Kaggle - Microsoft Cats vs Dogs dataset
Input validation	Decode image, remove invalid inputs, normalize to RGB
Image size	224 x 224 pixels, 3 color channels
Split	Train 19,998 / validation 2,500 / test 2,500
Method	Stratified 80% / 10% / 10%, random seed 42
Training augmentation	Random horizontal flip, +/-10 degree rotation, color jitter

3.2 Baseline model

The baseline is a compact SimpleCNN implemented in PyTorch. Three convolutional blocks apply convolution, batch normalization, ReLU activation, and max pooling. Adaptive average pooling removes dependence on the final spatial dimensions; dropout and a two-output linear layer produce class logits. This is intentionally lightweight enough for CPU inference and CI verification while remaining a credible image baseline.

Training parameter	Configured value
Epochs	10
Optimizer	Adam
Learning rate	0.001
Batch size	32
Loss	Cross entropy
Serialized release format	PyTorch state dictionary (.pt) plus JSON metadata

3.3 Experiment tracking and artifact lineage

MLflow stores the training and evaluation runs, including eight parameters, 44 metric records, and six artifacts. The training run ID is 58ae81476bf847deb788a3cde386325c and the evaluation run ID is 17762bf59c5a4bec877349b8695d5a55. The promoted model is protected by a SHA-256 checksum so the service and CI can detect missing or altered artifacts before readiness.

Promoted model SHA-256

9697741dfc20759f7cf04f829b3abf6c3b1a50bc93bfca3ae45891890fa191b7

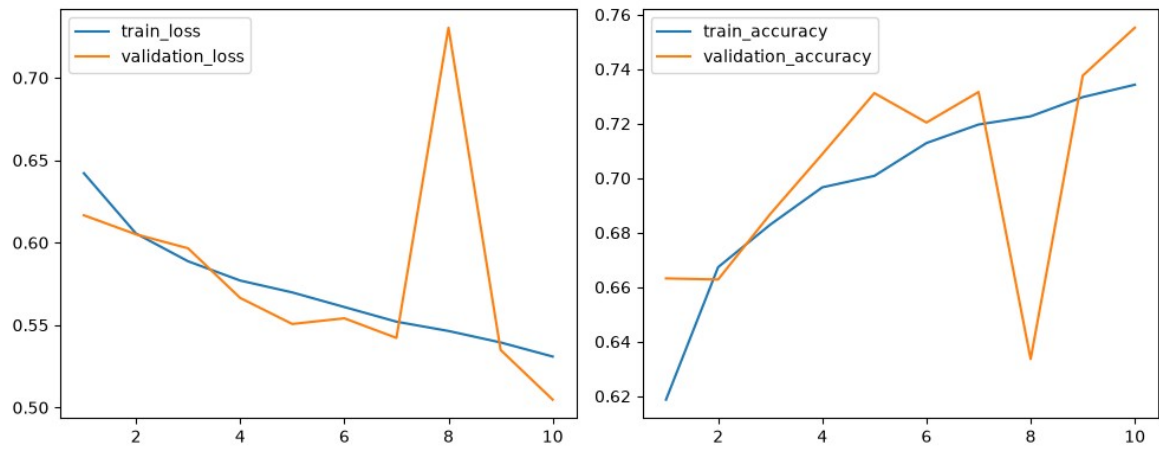


Figure 1. Training and validation loss/accuracy recorded across 10 epochs.

3.4 Held-out test performance

Metric	Result
Accuracy	0.7548
Precision	0.7613
Recall	0.7424
F1 score	0.7517
Best validation accuracy	0.7552
Confusion matrix	[[959, 291], [322, 928]]

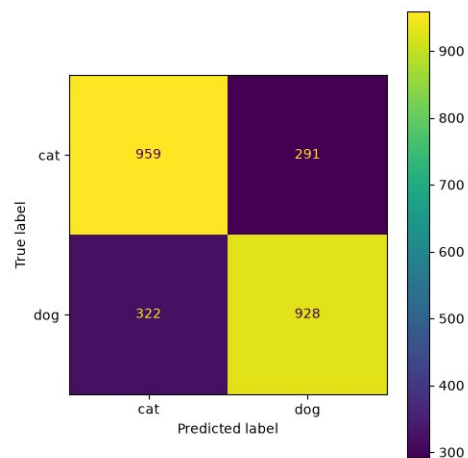


Figure 2. Confusion matrix on 2,500 held-out images (rows: true class; columns: predicted class).

4. M2 - Model Packaging and Containerization

4.1 Inference service

The application factory loads configuration from `params.yaml` and initializes the release model during startup. Readiness is deliberately stricter than liveness: `/health` shows that the process is running, while `/ready` confirms that the verified model is loaded.

Endpoint	Method	Purpose
<code>/health</code>	GET	Liveness check for process and container health.
<code>/ready</code>	GET	Readiness check; returns 200 only when the model is usable.
<code>/predict</code>	POST	Accepts an image and returns label, confidence, both probabilities, and model version.
<code>/metrics</code>	GET	Prometheus-compatible counters and latency histograms.

4.2 Reproducible environment

Key runtime and ML dependencies are version-pinned. Separate runtime, development, and CPU-specific requirement files keep the service image focused while preserving a complete developer/test environment. A central `params.yaml` captures model, preprocessing, training, data path, and serving configuration.

4.3 Container design

- Uses a Python 3.11 slim base and CPU-only PyTorch dependencies.
- Copies only the runtime package, configuration, promoted model, and release metadata.
- Runs the service as a non-root application user.
- Verifies the release model checksum and exposes an image health check against `/ready`.
- Starts Uvicorn on port 8000 with unbuffered logging for container observability.

```
docker build -t cats-dogs-mlops:local .
docker run --rm -p 8000:8000 --name cats-dogs-ci cats-dogs-mlops:local
curl --fail http://localhost:8000/ready
```

5. M3 - CI: Build, Test, and Image Publication

5.1 Automated test coverage

The suite contains unit tests for preprocessing and model/inference utilities plus API integration tests. The release integration test loads the real promoted artifact, checks readiness, submits a valid image, and validates the prediction contract.

Test group	Count	Coverage focus
API integration	6	Health, readiness, prediction validation, metrics, real release model
Inference utility	2	Probability/label conversion and inference behavior

Test group	Count	Coverage focus
Model utility	2	Model output shape and architectural behavior
Preprocessing	6	RGB conversion, resize/transform, validation, deterministic behavior
Total	16	All passed in the verified GitHub Actions run

5.2 Continuous-integration gates

1. Check out the repository and configure Python 3.11.
2. Install pinned development dependencies.
3. Run static checks and byte-code compilation.
4. Run pytest and publish a JUnit XML report.
5. Validate promoted-model metadata and checksum integrity.
6. Build the Docker image and start a named CI container.
7. Wait for readiness, exercise one real prediction, and fail on any smoke-test error.
8. Upload diagnostic reports/logs so failures remain inspectable.

5.3 Verified CI and registry result

Evidence	Verified result
Source commit	f06cd125d8625328df676cc5d38d2c9a7f1487ab
GitHub Actions run	33319484015
Tests	16 passed; pytest report uploaded
Container smoke test	Health/readiness and prediction checks passed
Registry	ghcr.io/praveen-kanwar-bits/cats-dogs-mlops-pipeline
Published tags	f06cd125d8625328df676cc5d38d2c9a7f1487ab and latest
Image digest	sha256:d360ec75dcf8863f5bb6cf54b6fbb77a3662faf24e58c7061eff202775b7776f

Why immutable tags matter

The deployment uses the exact commit-SHA image, not only latest. This preserves the link between source, CI evidence, container contents, and the deployed model release.

6. M4 - Continuous Deployment

6.1 Deployment target

Docker Compose is the selected target. The deployment manifest defines the image, port mapping, restart behavior, environment configuration, health check, and model service. This choice satisfies the assignment while keeping the demonstration reproducible on a single VM or self-hosted GitHub Actions runner.

6.2 CD workflow

1. Trigger after the CI workflow completes successfully for the main branch.
2. Resolve the immutable image tag from the successful source commit.
3. Authenticate the self-hosted runner to GitHub Container Registry.
4. Pull the new image and update the Compose service.
5. Wait for the readiness endpoint and submit one prediction request.
6. Fail the deployment workflow if either smoke check fails and retain service logs for diagnosis.

```
IMAGE_TAG=f06cd125d8625328df676cc5d38d2c9a7f1487ab
docker compose -f deployment/docker-compose.yml pull
docker compose -f deployment/docker-compose.yml up -d
python scripts/smoke_test.py --base-url http://localhost:8000 --retries 30 --delay 2
```

6.3 Deployment verification status

Implemented and ready for demonstration

The CD workflow, Docker Compose manifest, image-pull/update steps, health check, prediction smoke test, and failure diagnostics are present in the repository. A self-hosted runner must be online for a real target deployment. The final screen recording should show the deployment job or an equivalent local Compose update and live prediction.

7. M5 - Monitoring, Logging, and Post-Deployment Tracking

7.1 Operational telemetry

The service emits structured JSON logs for lifecycle events, requests, outcomes, model predictions, status codes, and elapsed time without logging raw image contents. A request ID supports correlation between API events and prediction results.

Metric	Meaning
http_requests_total	Requests grouped by method, path, and status
http_request_duration_seconds	Request latency histogram
prediction_requests_total	Prediction calls grouped by outcome
model_predictions_total	Predicted class distribution

Metric	Meaning
model_loaded	Binary gauge indicating model availability

7.2 Post-deployment performance

A simulated labeled batch uses 20 distinct images (10 cats and 10 dogs) to exercise the deployed prediction path and compare responses with true labels. The recorded local verification produced the following results for model version local-verified.

Metric	Result
Sample count	20
Accuracy	0.70
Precision	0.70
Recall	0.70
F1 score	0.70
Confusion matrix	[[7, 3], [3, 7]]

The small-batch score is not a replacement for the 2,500-image held-out evaluation. Its purpose is to prove that labels and predictions can be collected after deployment and compared over time. A production system would retain consented ground truth, add rolling windows, and alert on sustained quality or class-distribution changes.

8. Verification and Reproducibility

8.1 Reproduce the workflow

1. Create a Python 3.11 environment and install the pinned development requirements.
2. Configure the Kaggle credentials, download the dataset, and use DVC to reproduce versioned data stages.
3. Run preprocessing, training, evaluation, and model-promotion commands; inspect the MLflow runs and evaluation artifacts.
4. Run pytest and the release integrity check locally.
5. Build and start the container, then execute the smoke-test command.
6. Use Docker Compose for the deployment target and simulate labeled traffic for post-deployment evaluation.

```
python -m pytest -v --junitxml=pytest-report.xml
python scripts/smoke_test.py --base-url http://localhost:8000 --retries 30 --delay 2
python scripts/post_deployment_evaluation.py --base-url http://localhost:8000
```

8.2 Controls that protect reproducibility

- Deterministic data split seed and explicit preprocessing configuration.
- DVC manifests and checksums for raw/processed dataset state.

- Pinned key dependencies and a parameter file committed with the source.
- MLflow run identifiers and persisted evaluation artifacts.
- Release metadata that binds model architecture, class mapping, input size, version, and SHA-256 checksum.
- Immutable commit-SHA container tags and image digest evidence.
- Automated tests, model integrity checks, and post-build/post-deploy smoke tests.

9. Rubric Compliance Matrix

Module	Implemented evidence	Location
M1 - 10M	Git/DVC, preprocessing, SimpleCNN, model artifact, MLflow runs, plots	dvc.yaml, params.yaml, scripts/, artifacts/, mlruns/
M2 - 10M	FastAPI endpoints, pinned requirements, Dockerfile, local smoke path	src/, requirements*.txt, Dockerfile
M3 - 10M	16 tests, CI gates, Docker build, GHCR publication	tests/, .github/workflows/ci.yml
M4 - 10M	Compose target, main-branch CD, pull/update, health and prediction smoke tests	deployment/, .github/workflows/cd.yml
M5 - 10M	JSON logging, Prometheus metrics, simulated labeled batch, consolidated deliverables	src/, scripts/, docs/, dist/

Submission safeguard

This DOCX/PDF report is a required, first-class deliverable. The submission builder checks for both formats before producing the final ZIP so the report cannot be omitted silently.

10. Limitations and Recommended Next Steps

- The compact CNN is a baseline; transfer learning (for example MobileNetV3 or ResNet18) is likely to improve accuracy while preserving practical inference latency.
- The 20-image post-deployment batch demonstrates the mechanism but is too small for statistical assurance; add larger labeled windows and confidence intervals.
- The CD workflow depends on an online self-hosted runner. Capture one successful deployment run and live endpoint result in the required screen recording.
- For production use, add TLS, authentication, image-size limits at the edge, vulnerability scanning, signed images, and a rollback policy.
- Add drift dashboards and alerts after a stable stream of consented labeled outcomes becomes available.

11. Conclusion

The project implements every technical stage requested by the assignment: versioned data and code, reproducible model development, experiment tracking, a serialized model, a containerized REST service, automated tests, CI-based image creation and publication, a Docker Compose CD path with smoke tests, and operational/model monitoring. The verified CI and publication results provide traceable evidence

from source commit to registry digest. This formal report, the consolidated ZIP, and the <5-minute workflow recording together form the intended final submission package.

Appendix A - Final Deliverables

Deliverable	Expected submission item	Status at report generation
Source and configuration	Repository plus consolidated assignment ZIP	Prepared by submission builder
Trained model	Promoted .pt model and release metadata	Included in repository/ZIP
Formal report	MLOps_Assignment_2_Final_Report.docx and .pdf	Included in repository/ZIP
CI/CD evidence	Successful workflow logs and registry tag/digest	Verified for CI and image publication
Demonstration	Screen recording under 5 minutes	Record/upload separately using docs/DEMO_SCRIPT.md

Before upload, open the ZIP and confirm that both report files, the trained model, release metadata, DVC configuration, workflows, Dockerfile, deployment manifest, tests, and source package are present. Upload the recording alongside the ZIP if the learning-management system accepts multiple files; otherwise place the recording inside the archive only if the final size remains within the platform limit.

Appendix B - Key References

- Project repository: <https://github.com/praveen-kanwar-bits/cats-dogs-mlops-pipeline>
- Dataset: <https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset>
- Container image: ghcr.io/praveen-kanwar-bits/cats-dogs-mlops-pipeline
- Internal architecture and workflow documentation: docs/ARCHITECTURE.md and docs/MLOPS_WORKFLOW.md
- Evaluator traceability: docs/RUBRIC_TRACEABILITY.md and docs/EVALUATOR_REPORT.md

Appendix C - Evidence Summary

Evidence type	Identifier/value
Source commit	f06cd125d8625328df676cc5d38d2c9a7f1487ab
GitHub Actions run	33319484015
Training run	58ae81476bf847deb788a3cde386325c
Evaluation run	17762bf59c5a4bec877349b8695d5a55
Release model checksum	9697741dfc20759f7cf04f829b3abf6c3b1a50bc93bfca3ae45891890fa191b7
Published image digest	sha256:d360ec75dcf8863f5bb6cf54b6fbb77a3662faf24e58c7061eff202775b7776f